

Pandas Tutorial 6: Merge, Join, and Concat

In this tutorial, students will learn how to combine multiple DataFrames. In real world projects, data is often stored in different tables. To perform proper Exploratory Data Analysis, we must combine them correctly.

1. Why We Need to Combine DataFrames

In practical scenarios:

- One file may contain student information.
- Another file may contain student marks.
- Another file may contain attendance.

To perform full analysis, we combine these tables using merge, join, or concat.

2. Understanding Merge

The merge function is used to combine DataFrames based on a common column. It works similar to SQL joins.

Basic syntax:

```
pd.merge(df1, df2, on="ColumnName")
```

Example:

```
df1 = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["Ali", "Sara", "Ahmed"]
})

df2 = pd.DataFrame({
    "ID": [1, 2, 3],
    "Marks": [85, 90, 88]
})

merged_df = pd.merge(df1, df2, on="ID")
```

Here, ID is the common column used to combine both DataFrames.

3. Types of Merge

There are four main types of merge.

Inner Merge

```
pd.merge(df1, df2, on="ID", how="inner")
```

Returns only matching rows from both DataFrames.

Left Merge

```
pd.merge(df1, df2, on="ID", how="left")
```

Returns all rows from left DataFrame and matching rows from right DataFrame.

Right Merge

```
pd.merge(df1, df2, on="ID", how="right")
```

Returns all rows from right DataFrame and matching rows from left DataFrame.

Outer Merge

```
pd.merge(df1, df2, on="ID", how="outer")
```

Returns all rows from both DataFrames. Missing values will appear where there is no match.

4. Merge on Different Column Names

Sometimes column names are different in both DataFrames.

Example:

```
pd.merge(df1, df2, left_on="StudentID", right_on="ID")
```

This allows merging using different column names.

5. Understanding Join

The join method is used to combine DataFrames based on index.

Basic syntax:

```
df1.join(df2)
```

By default, join works on index.

If needed, we can specify join type:

```
df1.join(df2, how="left")
```

Join is useful when index already represents the relationship.

6. Understanding Concat

Concat is used to combine DataFrames either vertically or horizontally.

Vertical Concatenation

This adds rows.

```
pd.concat([df1, df2])
```

Horizontal Concatenation

This adds columns.

```
pd.concat([df1, df2], axis=1)
```

axis=0 means rows. axis=1 means columns.

7. Reset Index After Concat

After concatenation, index may repeat.

To fix this:

```
pd.concat([df1, df2], ignore_index=True)
```

This creates a new continuous index.

8. Difference Between Merge, Join, and Concat

Merge

Used to combine DataFrames based on common columns.

Join

Used to combine based on index.

Concat

Used to stack DataFrames vertically or horizontally.

Understanding the difference is important for correct data integration.

9. Conclusion of Tutorial 5

In this tutorial, students have learned:

1. Why combining DataFrames is important.
2. How to use merge.
3. Different types of joins in merge.
4. How to use join method.
5. How to use concat vertically and horizontally.
6. The difference between merge, join, and concat.

Now students can combine multiple datasets in real world projects.

In the next tutorial, we will learn about working with dates, strings, and advanced data cleaning techniques.

This completes Pandas Tutorial 5.